

Interpretable Dyna-Q for Maintenance Intervention Timing on NASA CMAPSS FD001

Anna Marie Benzon

AI 322 Reinforcement Learning Mini-Project

Abstract

This paper studies aircraft-engine maintenance intervention timing as a compact tabular reinforcement learning problem using the NASA CMAPSS FD001 turbofan degradation data set. The task is intentionally framed as an intervention-timing benchmark rather than a full repair simulator because CMAPSS contains run-to-failure trajectories but no counterfactual post-maintenance trajectories. A leakage-free 80/20 unit split was constructed from the FD001 training set. Sensor selection and normalization were fit only on training units, producing a 24-state abstraction based on health bucket, degradation-rate bucket, and anomaly flag. Four policies were evaluated: a rule-based baseline, SARSA, Q-learning, and Dyna-Q with planning steps $n \in \{0, 5, 10, 20\}$. The best three planning settings, $n = 0, 5, 10$, tied on final held-out reward at $+5.00$ with zero failures, while $n = 20$ degraded to -20.20 average reward and 20% failure rate. Planning improved greedy convergence speed: Dyna-Q with $n = 10$ reached the break-even threshold at 266.7 ± 28.9 episodes compared with 400.0 ± 86.6 for $n = 0$. In the final comparison, Q-learning and Dyna-Q achieved the same optimal held-out policy, SARSA intervened prematurely in all evaluation episodes, and the rule-based policy was safe but overly early. A safety override was redundant for the learned policy, reducing reward from $+5.00$ to -4.95 without lowering failures. Five-seed sensitivity analysis showed that the learned timing policy was fragile under harder seeds and reward/discretization variants. The main finding is therefore balanced: compact tabular RL can learn an interpretable FD001 maintenance-timing policy, and Dyna-Q provides a measurable sample-efficiency gain through faster greedy-policy convergence, although it does not improve final policy quality over Q-learning and the learned policy remains fragile under broader sensitivity tests.

Index Terms

CMAPSS, Dyna-Q, reinforcement learning, predictive maintenance, maintenance timing, Q-learning, SARSA, turbofan degradation.

I. INTRODUCTION

A. Significance of the Study

Aircraft engines are safety-critical assets whose degradation must be detected before failure. Maintenance decisions are sequential: continuing operation preserves useful life but increases late-failure risk, while early intervention avoids failure but wastes remaining life and incurs avoidable cost. This tradeoff makes maintenance intervention timing a natural reinforcement learning (RL) problem.

The NASA Commercial Modular Aero-Propulsion System Simulation (CMAPSS) data set is widely used in prognostics and health-management research because it provides simulated turbofan degradation trajectories [1], [2]. In the FD001 subset, each training engine is observed from normal operation until failure under one operating condition and one high-pressure compressor degradation mode. These complete run-to-failure histories make FD001 suitable for studying when an agent should stop operating and trigger maintenance.

This project builds on earlier statistical evidence that FD001 sensor readings contain meaningful degradation structure. Instead of estimating remaining useful life (RUL) directly, the present work asks whether a small and auditable tabular RL agent can learn the timing of maintenance intervention from sensor-derived states. The focus is interpretability and reproducibility, not maximum predictive accuracy.

B. Problem Statement

The general problem is to determine whether a compact tabular RL policy can make sensible terminal maintenance decisions before aircraft-engine failure. More specifically, this study asks:

- 1) Can a sensor-derived 24-state abstraction support a useful maintenance-timing policy on CMAPSS FD001?
- 2) Does Dyna-Q improve final policy quality or sample efficiency relative to ordinary tabular Q-learning?
- 3) How do Q-learning, SARSA, Dyna-Q, and a rule-based threshold policy differ in reward, failure rate, premature intervention, and intervention cycle?
- 4) Does a simple safety override reduce failures, or does it merely add inspection cost?
- 5) Are the conclusions stable under additional random seeds, reward scaling, and discretization changes?

C. Objectives

The objectives of the study are:

- 1) To prepare a leakage-free FD001 train/evaluation split by engine unit.
- 2) To construct an interpretable state representation from selected degradation-sensitive sensors.
- 3) To formulate a finite-state maintenance timing Markov decision process (MDP) with terminal maintenance actions.
- 4) To compare SARSA, Q-learning, and Dyna-Q against a rule-based baseline.
- 5) To evaluate Dyna-Q planning steps $n \in \{0, 5, 10, 20\}$.
- 6) To test whether a safety override improves evaluation outcomes.
- 7) To report sensitivity and limitations transparently.

D. Scope and Limitations

The scope is limited to FD001 and tabular RL. FD002–FD004, deep RL, fleet scheduling, probabilistic RUL modeling, and post-repair state simulation are outside the mini-project scope. This restriction is important because CMAPSS does not contain actual repair, overhaul, or replacement trajectories. Therefore, maintenance actions are modeled as terminal intervention decisions rather than transitions to a repaired engine state.

Several limitations follow from this design. First, the state is an approximate Markov representation derived from sensor summaries, not the true hidden engine condition. Second, correlation-based sensor selection captures mainly linear sensor–RUL relationships and may miss nonlinear degradation patterns. Third, reward values are benchmark costs rather than real airline maintenance economics. Fourth, the primary evaluation uses a held-out split from the complete FD001 training trajectories; the official test set is truncated and is reserved for secondary analysis.

II. BRIEF REVIEW OF RELATED LITERATURE

The theoretical foundation is tabular temporal-difference control. Sutton and Barto describe SARSA as an on-policy method, Q-learning as an off-policy method, and Dyna as an architecture that combines direct learning from real experience with planning updates from a learned model [3]. Dyna-Q is relevant when real trajectories are limited because planning can reuse observed transitions.

Maintenance optimization has also been studied through model-based RL. Zhang, Zhu, and Xie proposed a model-based RL approach for maintenance optimization in degrading systems with large state spaces [4]. Their work supports the use of planning-based RL for maintenance, although their setting is not the FD001 intervention-timing benchmark used here.

Recent aircraft-maintenance work has tended to use richer prognostics and deep RL. Lee and Mitici integrated probabilistic RUL prognostics with deep RL for predictive aircraft maintenance [5]. Tseremoglou and Santos studied aircraft-fleet condition-based maintenance under partial observability using a two-stage framework with POMDP/POMCP reasoning and DQN-based scheduling [6]. Wei et al. proposed a multi-agent RL approach for aero-engine maintenance optimization [7]. Reviews by Ogunfowora and Najjaran [8] and Zhang et al. [9] show that RL-based maintenance planning is an active area.

The gap addressed by this mini-project is narrower. Rather than claiming novelty for RL in aircraft maintenance, the study provides a small, reproducible, interpretable FD001 benchmark tied directly to core tabular RL methods. The intended contribution is an honest empirical account of what Dyna-Q does and does not add in this compact setting.

III. METHODOLOGY

A. Data Source and Preparation

The data source is NASA CMAPSS FD001. Each row contains an engine unit identifier, cycle number, three operating settings, and 21 sensor measurements. The complete training file contains 100 run-to-failure engine trajectories. RUL for a training row was computed as

$$RUL_{jt} = T_j - t, \quad (1)$$

where T_j is the final observed cycle for engine j and t is the current cycle.

The 100 FD001 training engines were split by unit into 80 training engines and 20 held-out evaluation engines using seed 42. The split was performed before feature engineering. Sensor selection and normalization statistics were fit on the 80 training engines only and then applied to both training and held-out engines. This prevents evaluation-unit information from leaking into preprocessing.

The five selected sensors were the nonconstant sensors with largest absolute Pearson correlation with RUL on the training units: s11, s4, s12, s7, and s15. Each selected sensor was standardized with training-set statistics. Sensor directions were aligned so higher transformed values corresponded to worse degradation, and the aligned standardized values were averaged into a composite health index.

TABLE I
FD001 DATA AND STATE CONSTRUCTION SUMMARY

Item	Value
Complete FD001 training engines	100
Training engines used for learning	80
Held-out evaluation engines	20
Selected sensors	s11, s4, s12, s7, s15
Health buckets	4
Degradation-rate buckets	3
Anomaly flags	2
Total tabular states	24

TABLE II
REWARD COMPONENTS USED IN THE MAINTENANCE ENVIRONMENT

Reward component	Value
Unexpected failure penalty	−100
Inspect cost	−1
Minor repair cost	−5
Major overhaul cost	−20
Replacement cost	−50
Timely intervention bonus in CRITICAL	+10
Late maintenance penalty in IMMINENT	−5
Premature heavy penalty in HEALTHY	−15
Premature minor penalty in HEALTHY	−8
WATCH replacement penalty	−5

B. State Representation

The state is defined as

$$s = (\text{health bucket}, \text{degradation bucket}, \text{anomaly flag}). \quad (2)$$

The health buckets are HEALTHY, WATCH, CRITICAL, and IMMINENT. The degradation-rate buckets are SLOW, MODERATE, and FAST. The anomaly flag is one if any selected standardized sensor has absolute value at least 2.0, and zero otherwise. The default health-index thresholds are -0.10 , 0.53 , and 1.35 ; the default degradation-rate thresholds are 0.0 and 0.05 . These choices produce a compact $4 \times 3 \times 2 = 24$ state space.

The abstraction is deliberately small so that Q-values and greedy actions can be inspected directly. However, it is only approximately Markovian. The health index and one-step degradation-rate feature summarize recent condition, but they do not fully represent latent engine physics.

C. Actions and Rewards

The action set contains five actions: CONTINUE, INSPECT, MINOR_REPAIR, MAJOR_OVERHAUL, and REPLACE. CONTINUE and INSPECT are nonterminal except in an IMMINENT state, where passive action leads to failure. MINOR_REPAIR, MAJOR_OVERHAUL, and REPLACE are terminal intervention actions. The reward function is cost-sensitive and intentionally transparent. Table II lists the benchmark reward components.

These values encode the intended benchmark preference: failure is much worse than maintenance cost, inspection is cheap, replacement is expensive, and the best terminal action is minor repair in the CRITICAL window. The highest per-episode reward available in the default configuration is therefore $+5$, obtained by choosing MINOR_REPAIR in a CRITICAL state.

D. RL Formulation and Update Equations

The maintenance timing task is modeled as an episodic discounted MDP,

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma), \quad (3)$$

where $\mathcal{S}$ is the 24-state health abstraction, $\mathcal{A}$ is the five-action maintenance set, $P(s'|s, a)$ is the transition law induced by the engine trajectory and terminal-action rules, R is the reward function in Table II, and $\gamma = 0.95$ is the discount factor. For an episode ending at time T , the discounted return from time t is

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k+1}. \quad (4)$$

The action-value function under policy π is

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]. \quad (5)$$

The optimal action-value function satisfies the Bellman optimality equation

$$Q^*(s, a) = \sum_{s'} P(s'|s, a) \left[R(s, a, s') + \gamma \max_{a'} Q^*(s', a') \right]. \quad (6)$$

During training, all learning agents use an ϵ -greedy behavior policy. If $a^* = \arg \max_b Q(s, b)$, then

$$\pi_\epsilon(a|s) = \begin{cases} 1 - \epsilon + \epsilon/|\mathcal{A}|, & a = a^*, \\ \epsilon/|\mathcal{A}|, & a \neq a^*. \end{cases} \quad (7)$$

All tabular methods use the same incremental update form

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \delta_t, \quad (8)$$

but differ in the temporal-difference (TD) target. For SARSA, the next action a_{t+1} is selected by the same ϵ -greedy behavior policy, giving the on-policy TD error

$$\delta_t^{\text{SARSA}} = r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t). \quad (9)$$

For Q-learning, the backup uses the greedy next-state value regardless of the exploratory behavior action:

$$\delta_t^{\text{Q}} = r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t). \quad (10)$$

For terminal transitions, the next-state value is set to zero, so the target is simply the terminal reward. Dyna-Q uses the same Q-learning TD error for real experience, then performs additional planning updates from a learned model. After observing a transition, the model stores it as

$$\hat{M}(s_t, a_t) \leftarrow \hat{M}(s_t, a_t) \cup \{(r_{t+1}, s_{t+1}, d_{t+1})\}, \quad (11)$$

where d_{t+1} is one for terminal transitions. For each planning step, a visited pair $(\tilde{s}, \tilde{a})$ is sampled uniformly, one stored outcome $(\tilde{r}, \tilde{s}', \tilde{d})$ is sampled from $\hat{M}(\tilde{s}, \tilde{a})$, and the planning update is

$$Q(\tilde{s}, \tilde{a}) \leftarrow Q(\tilde{s}, \tilde{a}) + \alpha \left[\tilde{r} + \gamma(1 - \tilde{d}) \max_{a'} Q(\tilde{s}', a') - Q(\tilde{s}, \tilde{a}) \right]. \quad (12)$$

Thus, $n = 0$ is equivalent to ordinary Q-learning, while $n > 0$ reuses previous experience through simulated updates.

E. Learning Algorithms

Four policies were evaluated:

- 1) a deterministic rule-based threshold policy;
- 2) SARSA, an on-policy tabular temporal-difference baseline;
- 3) Q-learning, an off-policy tabular temporal-difference baseline;
- 4) Dyna-Q, a Q-learning agent augmented with model-based planning.

For Dyna-Q, the learned model stores all observed transitions for each (s, a) pair rather than only the most recent transition. During planning, a previously visited state-action pair is sampled uniformly, and then a transition is sampled uniformly from the stored outcomes for that pair. This stochastic memory prevents planning from silently collapsing to the latest observed transition.

All learning agents used the common hyperparameters in Table III. Agents were trained for 500 episodes by sampling engines from the 80 training units. Greedy evaluation checkpoints were collected every 50 episodes on the held-out units.

TABLE III
TRAINING HYPERPARAMETERS

Parameter	Value
Learning rate α	0.10
Discount factor γ	0.95
Initial ϵ	0.20
Epsilon decay	0.995 per episode
Minimum ϵ	0.05
Training episodes	500
Main seeds	42, 7, 123
Sensitivity seeds	42, 7, 123, 0, 99
Planning steps	0, 5, 10, 20
Greedy evaluation interval	50 episodes

TABLE IV
EXPERIMENT 1: DYNA-Q PLANNING-STEP ABLATION

n	Reward	Fail	Prem.	Conv. ep.
0	5.00 ± 0.00	0.000	0.000	400 ± 87
5	5.00 ± 0.00	0.000	0.000	317 ± 29
10	5.00 ± 0.00	0.000	0.000	267 ± 29
20	-20.20 ± 43.65	0.200	0.000	400 ± 87

F. Safety Override

A simple deterministic safety layer was evaluated separately. If the learned policy chose CONTINUE or INSPECT in an IMMINENT state, the action was changed to MAJOR_OVERHAUL. If anomaly flag was one and the learned action was CONTINUE, the action was changed to INSPECT. The override was applied only during evaluation in the safety ablation, so the base learned policy remained inspectable.

G. Evaluation Metrics

The study reports average cumulative reward, failure rate, premature-any intervention rate, early-heavy intervention rate, mean intervention cycle, and greedy convergence. Premature-any intervention is any terminal maintenance action in HEALTHY or WATCH. Early-heavy intervention is MAJOR_OVERHAUL or REPLACE in HEALTHY. These additional metrics are necessary because total reward alone can hide unrealistic maintenance behavior.

IV. RESULTS AND DISCUSSION

A. Planning-Step Ablation

Table IV summarizes the Dyna-Q planning-step ablation. Dyna-Q with $n = 0$, $n = 5$, and $n = 10$ all achieved the same final held-out evaluation result: $+5.00$ average reward, zero failures, zero premature intervention, and mean intervention cycle 135.6. Since $n = 0$ is equivalent to ordinary Q-learning, the final-policy result is a null result for Dyna-Q superiority.

Planning did improve greedy convergence speed for moderate planning. The $n = 10$ agent reached the break-even greedy reward threshold at 266.7 ± 28.9 episodes, about 133 episodes earlier than $n = 0$. However, too much planning was harmful: $n = 20$ produced lower final reward and a 20% failure rate. These results suggest that planning can improve sample efficiency in the compact FD001 task, but it does not automatically improve final policy quality.

B. Algorithm Comparison

Table V compares the final policies. Q-learning and Dyna-Q with $n = 0$ achieved the same optimal held-out behavior. Both learned to avoid passive failure and to intervene in the CRITICAL window with MINOR_REPAIR, producing the maximum reward of $+5.00$.

The rule-based policy was safe but overly conservative, intervening prematurely in 70% of held-out episodes and acting earlier on average. SARSA avoided failure but acted prematurely in every evaluation episode, with a mean intervention cycle of only 2.1. This behavior shows why premature-any intervention is needed as a metric: early-heavy intervention alone would miss premature MINOR_REPAIR decisions.

Fig. 4 shows greedy convergence for SARSA, Q-learning, and Dyna-Q. SARSA had the best greedy area-under-curve among the learning agents but converged to an undesirable early-intervention policy. Q-learning and Dyna-Q had worse early greedy reward but reached the optimal CRITICAL-window policy by the end of training.



Fig. 1. Planning-step trade-off across final reward, failure behavior, and greedy threshold episode. Moderate planning improves convergence speed, while $n = 20$ increases failure risk and reduces final reward.

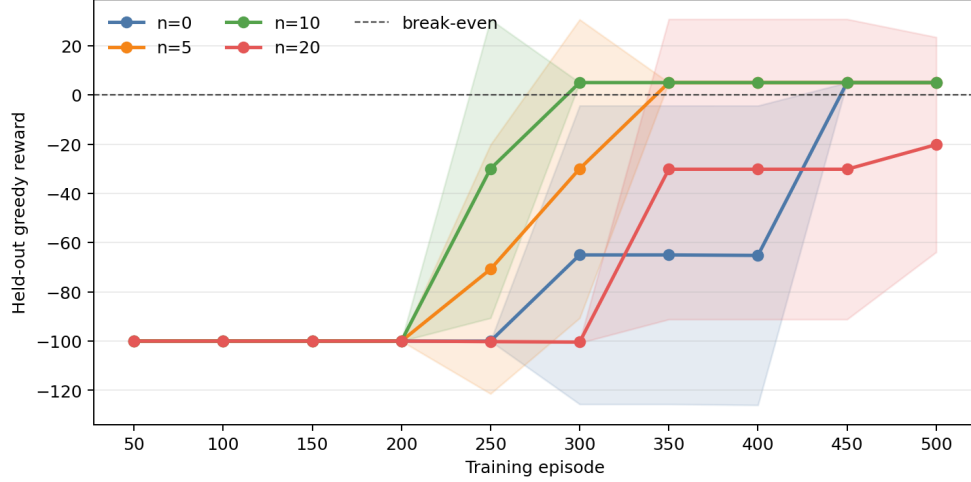


Fig. 2. Greedy reward convergence for Dyna-Q planning steps $n \in \{0, 5, 10, 20\}$ on the held-out FD001 evaluation engines.

TABLE V
EXPERIMENT 2: FINAL ALGORITHM COMPARISON

Policy	Reward	Fail	Prem.	Cycle
Rule-based	-16.45 ± 0.00	0.000	0.700	114.8
SARSA	-11.27 ± 0.46	0.000	1.000	2.1
Q-learning	5.00 ± 0.00	0.000	0.000	135.6
Dyna-Q	5.00 ± 0.00	0.000	0.000	135.6

TABLE VI
EXPERIMENT 3: SAFETY-OVERRIDE ABLATION

Condition	Reward	Fail	Prem.	Cycle
Base policy	5.00 ± 0.00	0.000	0.000	135.6
With override	-4.95 ± 0.00	0.000	0.000	135.6

C. Safety-Override Ablation

Table VI shows that the safety override did not reduce failures because the base learned policy already had zero failures. Instead, the override reduced reward from $+5.00$ to -4.95 . Trigger inspection showed that the IMMEDIATE passive-action rule fired zero times, while the anomaly CONTINUE-to-INSPECT rule fired 199 times across the 20 held-out units for each seed. The reward decrease is therefore attributable to unnecessary inspections.

This result is important because it prevents a misleading safety claim. The learned policy is safe on the held-out split without the override. The override is not harmful in failure terms, but it is costly and redundant under the trained policy.

Experiment 2: Policy quality and intervention behavior

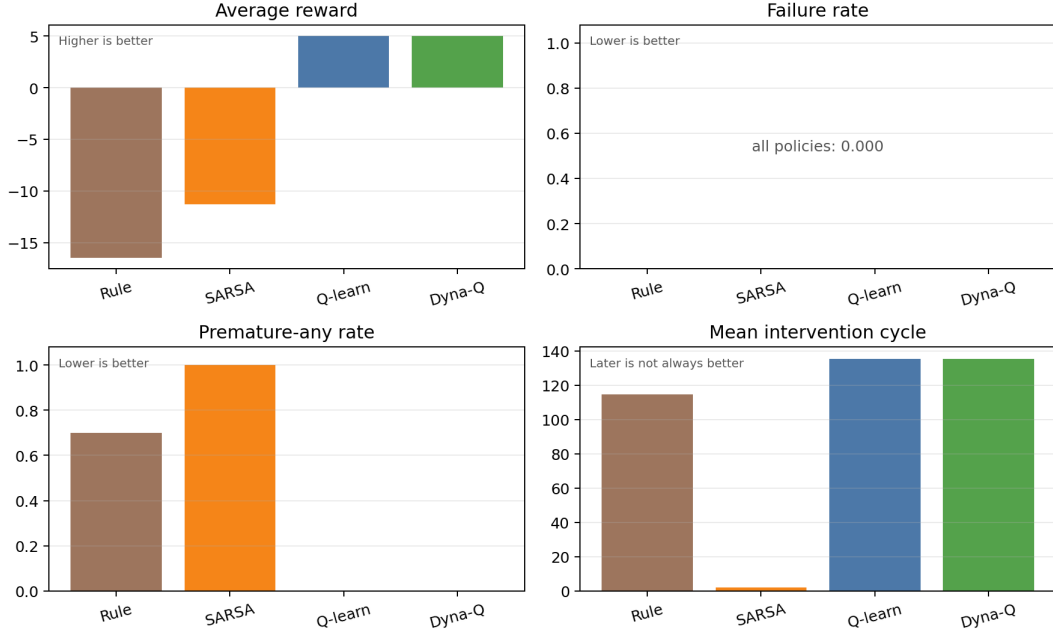


Fig. 3. Policy-quality and intervention-behavior metrics for the final algorithm comparison. Q-learning and Dyna-Q tie on reward and avoid premature intervention, while SARSA is failure-free but acts immediately.

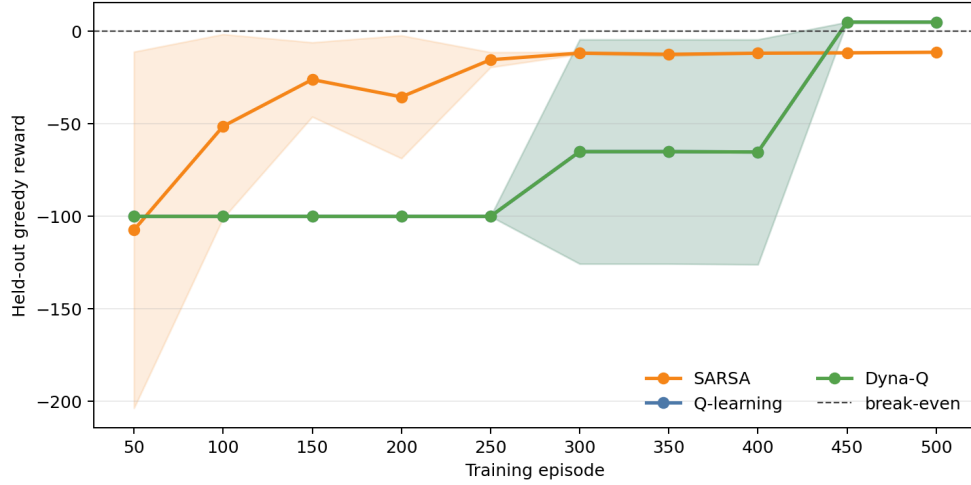


Fig. 4. Greedy reward convergence for SARSA, Q-learning, and Dyna-Q with the selected parsimonious setting $n = 0$.

TABLE VII
FIVE-SEED SENSITIVITY BASELINE

Policy	Reward	Fail	Prem.
Rule-based	-16.45 ± 0.00	0.000	0.700
SARSA	-11.39 ± 0.50	0.000	1.000
Q-learning	-9.78 ± 26.16	0.100	0.000
Dyna-Q	-9.78 ± 26.16	0.100	0.000

D. Sensitivity Analysis

The main three-seed result is strong but not robust. Table VII shows the default configuration under five seeds. Seeds 0 and 99 exposed non-convergence within 500 episodes, giving Q-learning and Dyna-Q a 10% failure rate and average reward -9.78 ± 26.16 . The rule-based policy remained failure-free but premature.

Experiment 3: Safety override is costly but not safer here

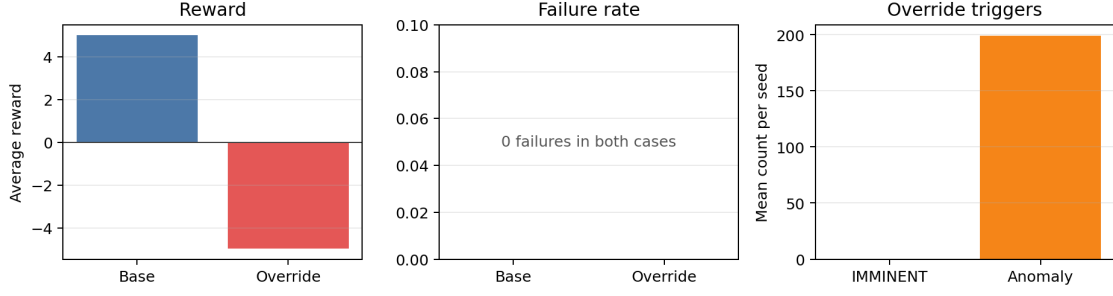


Fig. 5. Safety-override ablation. The override does not reduce failures because both conditions have zero failures, but it lowers reward through frequent anomaly-triggered inspections.

Eight sensitivity variants were then tested: two failure penalties, two action-cost scales, two CRITICAL/IMMINENT boundary settings, and two MODERATE/FAST boundary settings. The strict robustness criterion required Q-learning to beat the rule-based reward while also maintaining zero failure rate. Q-learning maintained that criterion in 0 of 8 variants. This does not erase the three-seed result, but it does mean the learned policy should be described as promising rather than robust.

Experiment 4: Sensitivity analysis across reward and bucket variants

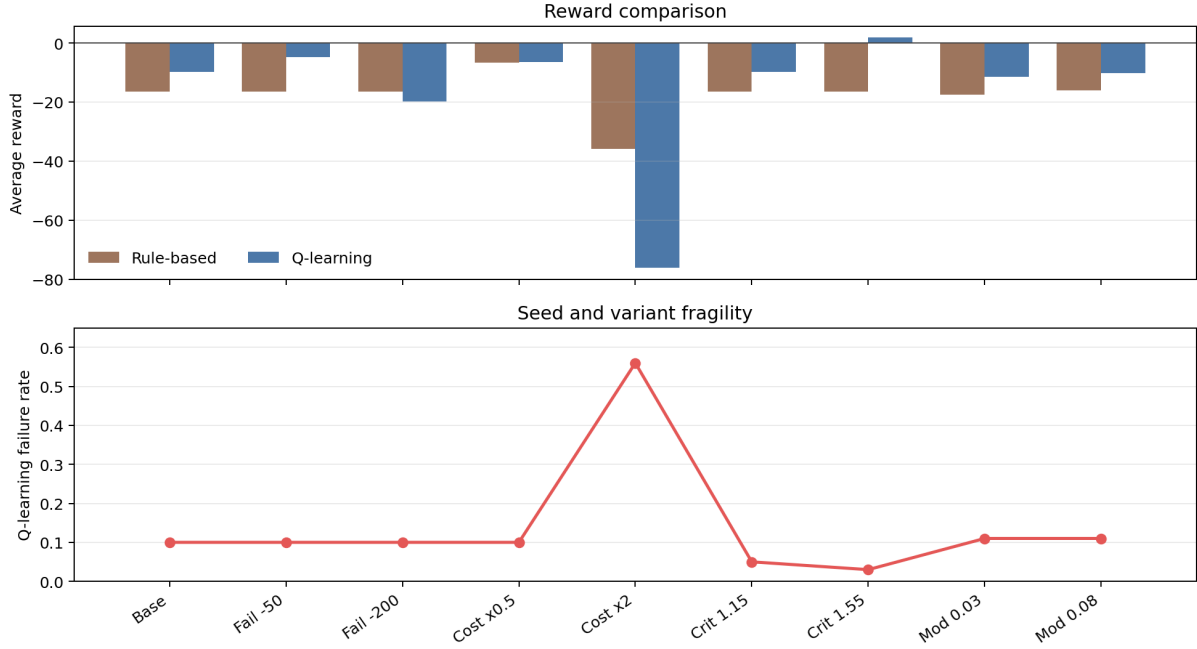


Fig. 6. Sensitivity analysis across reward and bucket variants. Q-learning often improves reward relative to the rule-based policy, but the failure-rate line shows fragility under additional seeds and changed cost or discretization settings.

E. Policy Inspection

The learned Q-table is interpretable because the state space is small. For Q-learning seed 42, HEALTHY and WATCH states generally map to CONTINUE, while the visited CRITICAL FAST state maps to MINOR_REPAIR. IMMINENT states were unvisited by the final greedy policy. The unvisited or tied CRITICAL SLOW/MODERATE entries should be interpreted as sparse-state artifacts: in the evaluated split, terminal intervention occurs after the policy reaches the CRITICAL window with a FAST trend, so those tied entries do not affect the reported greedy behavior. This supports the behavioral interpretation that the agent learns a CRITICAL-window terminal intervention rather than relying on late failure recovery.

TABLE VIII
REPRESENTATIVE GREEDY POLICY PATTERN FROM Q-LEARNING SEED 42

Health	Trend	Greedy action
HEALTHY	SLOW/MODERATE/FAST	CONTINUE
WATCH	SLOW/MODERATE/FAST	CONTINUE
CRITICAL	FAST	MINOR_REPAIR
CRITICAL	SLOW/MODERATE	unvisited/tied
IMMINENT	any	unvisited/tied

V. CONCLUSION AND RECOMMENDATIONS

This study formulated CMAPSS FD001 maintenance intervention timing as a compact tabular RL benchmark. The strongest held-out result is that Q-learning and Dyna-Q both learned a safe and interpretable CRITICAL-window maintenance policy with average reward +5.00, zero failures, and zero premature intervention under the main three-seed evaluation. The rule-based baseline was safe but intervened too early, while SARSA converged to a premature minor-repair policy.

The Dyna-Q result is balanced. Moderate planning provided a measurable sample-efficiency gain through faster greedy-policy convergence, with $n = 10$ reaching the threshold about 133 episodes earlier than $n = 0$. However, Dyna-Q did not improve final policy quality over Q-learning, and excessive planning at $n = 20$ degraded performance. The safety override was also not beneficial for the trained policy because failures were already zero; it mainly added inspection cost.

The main recommendation is to present the project as an interpretable maintenance-timing benchmark with an honest Dyna-Q null result, not as evidence that model-based RL dominates Q-learning. For future work, the policy should be tested on the official truncated FD001 test set, trained for more episodes under the harder seeds, and extended to FD002–FD004 only after the FD001 timing formulation is stable. A timing-only action space may also be worth testing because the current five-action reward makes MINOR_REPAIR in CRITICAL the dominant terminal action.

REFERENCES

- [1] A. Saxena, K. Goebel, D. Simon, and N. Eklund, “Damage propagation modeling for aircraft engine run-to-failure simulation,” in *Proc. Int. Conf. Prognostics and Health Management*, Denver, CO, USA, Oct. 2008, pp. 1–9, doi: 10.1109/PHM.2008.4711414.
- [2] A. Saxena and K. Goebel, “Turbofan engine degradation simulation data set,” NASA Prognostics Data Repository, NASA Ames Research Center, Moffett Field, CA, USA, 2008. [Online]. Available: <https://www.nasa.gov/intelligent-systems-division/discovery-and-systems-health/pcoe/pcoe-data-set-repository/>
- [3] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.
- [4] P. Zhang, X. Zhu, and M. Xie, “A model-based reinforcement learning approach for maintenance optimization of degrading systems in a large state space,” *Computers & Industrial Engineering*, vol. 161, Art. no. 107622, 2021, doi: 10.1016/j.cie.2021.107622.
- [5] J. Lee and M. Mitici, “Deep reinforcement learning for predictive aircraft maintenance using probabilistic remaining-useful-life prognostics,” *Reliability Engineering & System Safety*, vol. 230, Art. no. 108908, 2023, doi: 10.1016/j.ress.2022.108908.
- [6] I. Tseremoglou and B. F. Santos, “Condition-based maintenance scheduling of an aircraft fleet under partial observability: A deep reinforcement learning approach,” *Reliability Engineering & System Safety*, vol. 241, Art. no. 109582, 2024, doi: 10.1016/j.ress.2023.109582.
- [7] Z. Wei, Z. Zhao, Z. Zhou, and R. Yan, “Collaborative-sequential optimization for aero-engine maintenance based on multi-agent reinforcement learning,” *Expert Systems with Applications*, vol. 247, Art. no. 123358, 2024, doi: 10.1016/j.eswa.2024.123358.
- [8] O. Ogunfowora and H. Najjaran, “Reinforcement and deep reinforcement learning-based solutions for machine maintenance planning, scheduling policies, and optimization: A review,” *Journal of Manufacturing Systems*, vol. 70, pp. 244–263, 2023, doi: 10.1016/j.jmsy.2023.07.014.
- [9] Q. Zhang, Y. Liu, Y. Xiang, and T. Xiahou, “Reinforcement learning in reliability and maintenance optimization: A tutorial,” *Reliability Engineering & System Safety*, vol. 251, Art. no. 110401, 2024, doi: 10.1016/j.ress.2024.110401.